

TRABAJO PRÁCTICO

Arquitectura de Microservicios

E-Commerce

Materia: Construcción de Aplicaciones informáticas

1. Descripción General

En este trabajo práctico los alumnos deberán diseñar e implementar un sistema de E-Commerce basado en una arquitectura de microservicios, exponiendo cada funcionalidad como una REST API independiente en C# con .NET Core 8.

El sistema contemplará lógica de negocio y aspectos transversales: documentación de APIs, manejo estructurado de errores con códigos propios, monitoreo y observabilidad.

Persistencia: Provista por la cátedra como librería. Los alumnos únicamente invocan sus métodos desde la capa de servicios.

2. Objetivos de Aprendizaje

- Diseñar e implementar microservicios con responsabilidades bien definidas.
- Exponer REST APIs siguiendo buenas prácticas (verbos HTTP, códigos de estado, recursos).
- Documentar APIs con Swagger / OpenAPI incluyendo ejemplos de request y response.
- Implementar manejo global de errores con `ExceptionHandler` y códigos de error propios.
- Incorporar logs estructurados y métricas básicas de observabilidad.
- Comprender los desafíos de comunicación HTTP entre microservicios.

3. Contrato de Errores

3.1. Estructura de respuesta de error

Todas las respuestas de error (4xx y 5xx) deben tener la siguiente estructura. Los campos `errorCode` y `errorMessage` son obligatorios y deben tomarse del catálogo definido en cada sección de API:

```
{
  "type": "https://tools.ietf.org/html/rfc7231#section-6.5.4",
  "title": "Not Found",
  "status": 404,
  "detail": "El recurso solicitado no fue encontrado.",
  "instance": "/api/products/99",
  "errorCode": "PRD-001",
  "errorMessage": "Producto no encontrado."
}
```

Para errores de validación con múltiples campos, `errorMessage` puede listar todos los problemas separados por punto y coma.

4. Requerimientos Funcionales y Contratos de API

4.1. Products API

Endpoints

Método	Endpoint	Descripción	HTTP Status posibles
GET	/api/products	Listar productos (filtro ?categoria= y ?nombre=)	200, 500
GET	/api/products/{id}	Obtener producto por ID	200, 404, 500
POST	/api/products	Crear nuevo producto	201, 400, 409, 500
PUT	/api/products/{id}	Actualizar producto existente	200, 400, 404, 500
DELETE	/api/products/{id}	Eliminar producto	204, 404, 409, 500

Categorías de productos

Nota: Las categorías son de carácter informativo. No es necesario validar que el valor ingresado pertenezca a esta lista.

Categoría	Ejemplos de productos
Electrónica	Notebooks, celulares, tablets, auriculares
Indumentaria	Ropa, calzado, accesorios de moda
Hogar y Deco	Muebles, iluminación, vajilla, textiles
Deportes	Equipamiento deportivo, ropa técnica, suplementos
Libros y Medios	Libros, revistas, música, software
Juguetes	Juegos de mesa, juguetes infantiles, videojuegos
Alimentos	Alimentos no perecederos, bebidas, snacks
Herramientas	Herramientas manuales, eléctricas, materiales de construcción
Salud y Belleza	Cosmética, medicamentos OTC, artículos de higiene
Otros	Productos que no encajan en las categorías anteriores

Catálogo de errores

errorCode	HTTP	errorMessage sugerido	Cuándo se devuelve
PRD-001	404	Producto no encontrado.	GET/PUT/DELETE cuando el ID no existe.
PRD-002	400	Los datos del producto son inválidos.	POST/PUT con campos faltantes o formato incorrecto.
PRD-003	409	Ya existe un producto con ese nombre en la categoría.	POST cuando se intenta crear un duplicado.
PRD-004	409	El producto tiene órdenes activas y no puede eliminarse.	DELETE cuando hay órdenes Pendiente o Confirmada que lo referencian.
PRD-005	500	Error interno al procesar el producto.	Error inesperado en servicio o persistencia.

Ejemplos de Request / Response

GET /api/products/{id} — Éxito

GET /api/products/{id}	Response 200 OK
Request Body	Response Body
(sin body)	{ "id": "3fa85f64-5717-4562-b3fc-2c963f66afa6", "nombre": "Notebook Dell XPS 15", "descripcion": "Laptop 15 pulgadas, 32GB RAM", "precio": 1500.00, "stock": 10, "categoria": "Electrónica", "fechaCreacion": "2024-01-15T10:30:00Z" }

GET /api/products/{id} — Error: producto no encontrado

GET /api/products/99 → 404 (PRD-001)
{ "type": "https://tools.ietf.org/html/rfc7231#section-6.5.4", "title": "Not Found", "status": 404, "detail": "El recurso solicitado no fue encontrado.", "instance": "/api/products/99", "errorCode": "PRD-001", "errorMessage": "Producto no encontrado." }

POST /api/products — Éxito

POST /api/products	Response 201 Created
--------------------	----------------------

Request Body	Response Body
<pre>{ "nombre": "Notebook Dell XPS 15", "descripcion": "Laptop 15 pulgadas, 32GB RAM", "precio": 1500.00, "stock": 10, "categoria": "Electrónica" }</pre>	<pre>{ "id": "3fa85f64-5717-4562-b3fc- 2c963f66afa6", "nombre": "Notebook Dell XPS 15", "descripcion": "Laptop 15 pulgadas, 32GB RAM", "precio": 1500.00, "stock": 10, "categoria": "Electrónica", "fechaCreacion": "2024-01-15T10:30:00Z" }</pre>

POST /api/products — Error: nombre duplicado en categoría

POST /api/products → 409 (PRD-003)
<pre>{ "type": "https://tools.ietf.org/html/rfc7231#section-6.5.9", "title": "Conflict", "status": 409, "detail": "Ya existe un recurso con esos datos.", "instance": "/api/products", "errorCode": "PRD-003", "errorMessage": "Ya existe un producto con ese nombre en la categoría 'Electrónica'." }</pre>

PUT /api/products/{id} — Éxito

PUT /api/products/{id}	Response 200 OK
Request Body	Response Body
<pre>{ "nombre": "Notebook Dell XPS 15", "descripcion": "Laptop 15 pulgadas, 64GB RAM", "precio": 1750.00, "stock": 8, "categoria": "Electrónica" }</pre>	<pre>{ "id": "3fa85f64-5717-4562-b3fc- 2c963f66afa6", "nombre": "Notebook Dell XPS 15", "descripcion": "Laptop 15 pulgadas, 64GB RAM", "precio": 1750.00, "stock": 8, "categoria": "Electrónica", "fechaCreacion": "2024-01-15T10:30:00Z" }</pre>

DELETE /api/products/{id} — Éxito

DELETE /api/products/{id}	Response 204 No Content
Request Body	Response Body
(sin body)	(sin body)

DELETE /api/products/{id} — Error: producto con órdenes activas

DELETE /api/products/{id} → 409 (PRD-004)

```
{
  "type": "https://tools.ietf.org/html/rfc7231#section-6.5.9",
  "title": "Conflict",
  "status": 409,
  "detail": "No se puede eliminar el recurso.",
  "instance": "/api/products/3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "errorCode": "PRD-004",
  "errorMessage": "El producto tiene órdenes activas y no puede eliminarse."
}
```

4.2. Users API

Endpoints

Método	Endpoint	Descripción	HTTP Status posibles
POST	/api/users/register	Registrar nuevo usuario	201, 400, 409, 500
POST	/api/users/login	Autenticar usuario (email + password)	200, 400, 401, 403, 500

Regla de bloqueo: Al acumular 3 IntentosFallidos consecutivos, Activo pasa a false. El campo PasswordHash nunca debe incluirse en ninguna respuesta.

Catálogo de errores

errorCode	HTTP	errorMessage sugerido	Cuándo se devuelve
USR-001	409	El email ya está registrado.	POST /register cuando el email ya existe.
USR-002	400	Los datos del usuario son inválidos.	POST /register con campos faltantes o formato incorrecto.
USR-003	401	Credenciales incorrectas.	POST /login cuando email o contraseña no coinciden.
USR-004	403	Usuario bloqueado por demasiados intentos fallidos.	POST /login con 3+ intentos fallidos acumulados.
USR-005	403	Usuario bloqueado por detección de fraude.	POST /login cuando el usuario fue bloqueado manualmente.
USR-006	500	Error interno al procesar el usuario.	Error inesperado en servicio o persistencia.

Ejemplos de Request / Response

POST /api/users/register — Éxito

POST /api/users/register

Response 201 Created

Request Body	Response Body
<pre>{ "nombre": "María", "apellido": "González", "email": "maria@email.com", "password": "MiPassword123!" }</pre>	<pre>{ "id": "a1b2c3d4-0000-0000-0000-111122223333", "nombre": "María", "apellido": "González", "email": "maria@email.com", "fechaRegistro": "2024-03-10T09:00:00Z", "activo": true }</pre>

POST /api/users/register — Error: email duplicado

POST /api/users/register → 409 (USR-001)
<pre>{ "type": "https://tools.ietf.org/html/rfc7231#section-6.5.9", "title": "Conflict", "status": 409, "detail": "Ya existe un recurso con esos datos.", "instance": "/api/users/register", "errorCode": "USR-001", "errorMessage": "El email 'maria@email.com' ya está registrado." }</pre>

POST /api/users/login — Éxito

POST /api/users/login	Response 200 OK
Request Body	Response Body
<pre>{ "email": "maria@email.com", "password": "MiPassword123!" }</pre>	<pre>{ "id": "a1b2c3d4-0000-0000-0000-111122223333", "nombre": "María", "apellido": "González", "email": "maria@email.com" }</pre>

POST /api/users/login — Error: credenciales incorrectas

POST /api/users/login → 401 (USR-003)
<pre>{ "type": "https://tools.ietf.org/html/rfc7235#section-3.1", "title": "Unauthorized", "status": 401, "detail": "Las credenciales no son válidas.", "instance": "/api/users/login", "errorCode": "USR-003", "errorMessage": "Credenciales incorrectas." }</pre>

POST /api/users/login — Error: usuario bloqueado por intentos fallidos

POST /api/users/login → 403 (USR-004)

```
{
  "type": "https://tools.ietf.org/html/rfc7231#section-6.5.3",
  "title": "Forbidden",
  "status": 403,
  "detail": "El acceso está prohibido.",
  "instance": "/api/users/login",
  "errorCode": "USR-004",
  "errorMessage": "Su cuenta fue bloqueada por superar el máximo de intentos fallidos. Contacte a soporte."
}
```

POST /api/users/login — Error: usuario bloqueado por fraude

POST /api/users/login → 403 (USR-005)

```
{
  "type": "https://tools.ietf.org/html/rfc7231#section-6.5.3",
  "title": "Forbidden",
  "status": 403,
  "detail": "El acceso está prohibido.",
  "instance": "/api/users/login",
  "errorCode": "USR-005",
  "errorMessage": "Su cuenta fue suspendida por razones de seguridad. Contacte a soporte."
}
```

4.3. Orders API

Endpoints

Método	Endpoint	Descripción	HTTP Status posibles
GET	/api/orders	Listar órdenes (filtro ?usuarioId=)	200, 500
GET	/api/orders/{id}	Obtener detalle de una orden	200, 404, 500
POST	/api/orders	Crear nueva orden	201, 400, 404, 409, 422, 500
PUT	/api/orders/{id}/status	Actualizar estado de la orden	200, 400, 404, 409, 500

Catálogo de errores

errorCode	HTTP	errorMessage sugerido	Cuándo se devuelve
ORD-001	404	Orden no encontrada.	GET/PUT cuando el ID de orden no existe.

ORD-002	400	Los datos de la orden son inválidos.	POST con campos faltantes o lista de items vacía.
ORD-003	404	Usuario no encontrado al crear la orden.	POST cuando el UsuarioId no existe en Users API.
ORD-004	404	Producto no encontrado al crear la orden.	POST cuando algún ProductId no existe en Products API.
ORD-005	422	Stock insuficiente para uno o más productos.	POST cuando la cantidad solicitada supera el stock disponible.
ORD-006	409	El estado de la orden no puede ser modificado.	PUT /status cuando la transición de estado no es válida.
ORD-007	500	Error interno al procesar la orden.	Error inesperado en servicio o persistencia.

Ejemplos de Request / Response

GET /api/orders/{id} — Éxito

GET /api/orders/{id}	Response 200 OK
Request Body	Response Body
(sin body)	{ "id": "f1e2d3c4-0000-0000-0000-aabbccddeeff", "usuarioId": "a1b2c3d4-0000-0000-0000-11112223333", "items": [{ "productoId": "3fa85f64-5717-4562-b3fc-2c963f66afa6", "cantidad": 2, "precioUnitario": 1500.00 }], "total": 3000.00, "estado": "Pendiente", "fechaCreacion": "2024-03-10T11:00:00Z" }

POST /api/orders — Éxito

POST /api/orders	Response 201 Created
Request Body	Response Body
{ "usuarioId": "a1b2c3d4-0000-0000-0000-11112223333", "items": [{ "productoId": "3fa85f64-5717-4562-b3fc-2c963f66afa6", 	{ "id": "f1e2d3c4-0000-0000-0000-aabbccddeeff", "usuarioId": "a1b2c3d4-0000-0000-0000-11112223333", "items": [{

<pre>{ "cantidad": 2 }</pre>	<pre>{ "productoId": "3fa85f64-5717-4562-b3fc-2c963f66afa6", "cantidad": 2, "precioUnitario": 1500.00 }, { "total": 3000.00, "estado": "Pendiente", "fechaCreacion": "2024-03-10T11:00:00Z" }</pre>
--------------------------------	---

POST /api/orders — Error: stock insuficiente

POST /api/orders → 422 (ORD-005)
<pre>{ "type": "https://tools.ietf.org/html/rfc4918#section-11.2", "title": "Unprocessable Entity", "status": 422, "detail": "No se puede procesar la solicitud.", "instance": "/api/orders", "errorCode": "ORD-005", "errorMessage": "Stock insuficiente para 'Notebook Dell XPS 15'. Disponible: 2, solicitado: 5." }</pre>

PUT /api/orders/{id}/status — Éxito

PUT /api/orders/{id}/status	Response 200 OK
Request Body	Response Body
<pre>{ "estado": "Confirmada" }</pre>	<pre>{ "id": "f1e2d3c4-0000-0000-0000-aabbccddeeff", "estado": "Confirmada", "fechaActualizacion": "2024-03-10T12:00:00Z" }</pre>

PUT /api/orders/{id}/status — Error: transición inválida

PUT /api/orders/{id}/status → 409 (ORD-006)
<pre>{ "type": "https://tools.ietf.org/html/rfc7231#section-6.5.9", "title": "Conflict", "status": 409, "detail": "No se puede modificar el estado.", "instance": "/api/orders/f1e2d3c4-0000-0000-0000-aabbccddeeff/status", "errorCode": "ORD-006", "errorMessage": "Una orden en estado 'Entregada' no puede volver a 'Pendiente'." }</pre>

4.4. Cart API

Endpoints

Método	Endpoint	Descripción	HTTP Status posibles
GET	/api/cart/{userId}	Obtener carrito del usuario	200, 404, 500
POST	/api/cart/{userId}/items	Agregar producto al carrito	200, 400, 404, 422, 500
PUT	/api/cart/{userId}/items/{productId}	Actualizar cantidad de un item	200, 400, 404, 422, 500
DELETE	/api/cart/{userId}/items/{productId}	Quitar un producto del carrito	204, 404, 500
DELETE	/api/cart/{userId}	Vaciar carrito completo	204, 404, 500

Catálogo de errores

errorCode	HTTP	errorMessage sugerido	Cuándo se devuelve
CRT-001	404	Carrito no encontrado.	GET/PUT/DELETE cuando el userId no tiene carrito activo.
CRT-002	404	Producto no encontrado.	POST/PUT cuando el ProductId no existe en Products API.
CRT-003	422	Stock insuficiente para agregar al carrito.	POST/PUT cuando la cantidad supera el stock disponible.
CRT-004	400	Cantidad inválida.	POST/PUT cuando la cantidad es menor o igual a cero.
CRT-005	500	Error interno al procesar el carrito.	Error inesperado en servicio o persistencia.

Ejemplos de Request / Response

GET /api/cart/{userId} — Éxito

GET /api/cart/{userId}	Response 200 OK
Request Body	Response Body
(sin body)	<pre>{ "usuarioId": "a1b2c3d4-0000-0000-0000-111122223333", "items": [{ "productId": "3fa85f64-5717-4562-b3fc-2c963f66afa6", "cantidad": 1 }, { "productId": "aaaabbbb-cccc-dddd-eeee-ffff00001111", "cantidad": 3 }], }</pre>

	<pre>"fechaActualizacion": "2024-03-10T10:45:00Z" }</pre>
--	---

POST /api/cart/{userId}/items — Éxito

POST /api/cart/{userId}/items	Response 200 OK
Request Body	Response Body
<pre>{ "productoId": "3fa85f64-5717-4562-b3fc-2c963f66afa6", "cantidad": 2 }</pre>	<pre>{ "usuarioId": "a1b2c3d4-0000-0000-0000-111122223333", "items": [{ "productoId": "3fa85f64-5717-4562-b3fc-2c963f66afa6", "cantidad": 2 }], "fechaActualizacion": "2024-03-10T10:50:00Z" }</pre>

POST /api/cart/{userId}/items — Error: stock insuficiente

POST /api/cart/{userId}/items → 422 (CRT-003)
<pre>{ "type": "https://tools.ietf.org/html/rfc4918#section-11.2", "title": "Unprocessable Entity", "status": 422, "detail": "No se puede procesar la solicitud.", "instance": "/api/cart/a1b2c3d4/items", "errorCode": "CRT-003", "errorMessage": "Stock insuficiente. Disponible: 1, solicitado: 5." }</pre>

PUT /api/cart/{userId}/items/{productId} — Éxito

PUT /api/cart/{userId}/items/{productId}	Response 200 OK
Request Body	Response Body
<pre>{ "cantidad": 4 }</pre>	<pre>{ "usuarioId": "a1b2c3d4-0000-0000-0000-111122223333", "items": [{ "productoId": "3fa85f64-5717-4562-b3fc-2c963f66afa6", "cantidad": 4 }], "fechaActualizacion": "2024-03-10T11:05:00Z" }</pre>

DELETE /api/cart/{userId}/items/{productId} — Éxito

DELETE /api/cart/{userId}/items/{productId}	Response 204 No Content
Request Body	Response Body
(sin body)	(sin body)

DELETE /api/cart/{userId} — Éxito

DELETE /api/cart/{userId}	Response 204 No Content
Request Body	Response Body
(sin body)	(sin body)

4.5. Notifications API

Endpoints

Método	Endpoint	Descripción	HTTP Status posibles
POST	/api/notifications/send	Registrar y simular envío de notificación	201, 400, 404, 500
GET	/api/notifications/{userId}	Listar notificaciones de un usuario	200, 404, 500

Catálogo de errores

errorCode	HTTP	errorMessage sugerido	Cuándo se devuelve
NTF-001	404	Usuario no encontrado.	POST cuando el UserId no existe en Users API.
NTF-002	400	Los datos de la notificación son inválidos.	POST con campos faltantes o tipo no reconocido.
NTF-003	404	No se encontraron notificaciones para el usuario.	GET cuando el userId no tiene notificaciones registradas.
NTF-004	500	Error interno al procesar la notificación.	Error inesperado en servicio o persistencia.

Ejemplos de Request / Response

POST /api/notifications/send — Éxito

POST /api/notifications/send	Response 201 Created
Request Body	Response Body
{ "userId": "a1b2c3d4-0000-0000-0000-111122223333",	{ "id": "11112222-3333-4444-5555-666677778888",

<pre>"mensaje": "Su orden #fle2d3c4 fue confirmada.", "tipo": "Email" }</pre>	<pre>"usuarioId": "a1b2c3d4-0000-0000-0000-111122223333", "mensaje": "Su orden #fle2d3c4 fue confirmada.", "tipo": "Email", "estado": "Enviada", "fechaEnvio": "2024-03-10T12:01:00Z" }</pre>
---	---

POST /api/notifications/send — Error: usuario no encontrado

POST /api/notifications/send → 404 (NTF-001)	
<pre>{ "type": "https://tools.ietf.org/html/rfc7231#section-6.5.4", "title": "Not Found", "status": 404, "detail": "El recurso solicitado no fue encontrado.", "instance": "/api/notifications/send", "errorCode": "NTF-001", "errorMessage": "El usuario destinatario no fue encontrado." }</pre>	

GET /api/notifications/{userId} — Éxito

GET /api/notifications/{userId}	Response 200 OK
Request Body	Response Body
<pre>(sin body)</pre>	<pre>[{ "id": "11112222-3333-4444-5555-666677778888", "usuarioId": "a1b2c3d4-0000-0000-0000-111122223333", "mensaje": "Su orden fue confirmada.", "tipo": "Email", "estado": "Enviada", "fechaEnvio": "2024-03-10T12:01:00Z" }]</pre>

5. Requerimientos No Funcionales

5.1. Documentación de APIs con Swagger / OpenAPI

- Exponer Swagger UI en /swagger en cada microservicio.
- Documentar cada endpoint con sus parámetros, body, y todos los posibles códigos de respuesta incluyendo los errorCode.
- Incluir en Swagger ejemplos de response de éxito y de error siguiendo los contratos de la sección 4.
- Usar XML comments en controladores y modelos. Agrupar endpoints por tags.

5.2. Manejo de Errores con IExceptionHandler

- Usar app.UseExceptionHandler() en Program.cs. No se permite middleware personalizado para este fin.
- Crear una excepción de dominio por tipo de error (NotFoundException, BusinessException, ValidationException).
- Cada IExceptionHandler construye la respuesta con errorCode y errorMessage del catálogo definido en la sección 4.
- No exponer stack traces. Controlar el nivel de detalle por entorno (appsettings.Development.json vs Production).

5.3. Logging con Serilog

- Sinks: consola (formato legible) y archivo (formato JSON estructurado).
- Incluir en cada log: Timestamp, Nivel, Servicio, Endpoint, Correlation ID, y errorCode cuando aplique.
- Loggear inicio/fin de cada request con duración. Errores de negocio como Warning, errores inesperados como Error.

5.4. Health Checks

- GET /health, GET /health/ready y GET /health/live en cada servicio.
- Respuesta JSON con estado: Healthy, Degraded o Unhealthy.

5.5. Correlation ID

- Generar X-Correlation-Id único por request y propagarlo en llamadas HTTP salientes.
- Incluirlo en todos los logs del request y como campo extra en las respuestas de error.

6. Estructura del Proyecto

ECommerce.sln

```
|— src/
|   |— Products.API/
|   |— Users.API/
|   |— Orders.API/
|   |— Cart.API/
|   └─ Notifications.API/
|— docs/
└─ README.md
```

Products.API/

```
|— Controllers/
|— Models/           # Entidades del dominio
|— DTOs/             # Request y Response DTOs
|— Services/         # Lógica de negocio
|— Exceptions/       # NotFoundException, BusinessException, etc.
|— ExceptionHandlers/ # IExceptionHandler por tipo de excepción
|— logs/             # Archivos de log de Serilog
└─ Program.cs
```

7. Tecnologías y Paquetes NuGet

Propósito	Paquete / Tecnología
Framework base	.NET Core 8 / ASP.NET Core
Persistencia	Librería provista por la cátedra
Documentación API	Swashbuckle.AspNetCore
Logging	Serilog + Serilog.Sinks.Console + Serilog.Sinks.File
Validación	Data Annotations (incluido en .NET)
Health Checks	Microsoft.Extensions.Diagnostics.HealthChecks
HTTP entre servicios	IHttpClientFactory (incluido en .NET)
Manejo de errores	ExceptionHandler + ProblemDetails (incluido en .NET 8)

8. Criterios de Evaluación

Criterio	Puntaje	Peso
Implementación y funcionalidad de los endpoints (HTTP codes correctos)	0-35	35%
Códigos de error propios (catálogo completo, errorCode y errorMessage en responses)	0-15	15%
Documentación Swagger con ejemplos de request/response y errores	0-15	15%
Manejo de errores global con IExceptionHandler	0-15	15%
Logging estructurado con Serilog y Correlation ID	0-12	12%
Health Checks y diseño del código / README	0-8	8%

9. Condiciones de Entrega

9.1. Repositorio

- Repositorio Git público o compartido con el docente.
- Todos los proyectos dentro de una única solución (.sln)..

9.2. Documentación entregable

- README.md con cómo ejecutar el proyecto..
- Diagrama de arquitectura.
- Capturas de Swagger UI mostrando ejemplos de response de error con errorCode y errorMessage.

9.3. Defensa

- Presentación grupal de 15 a 20 minutos.
- Demo en vivo: invocar endpoints exitosos y de error desde Swagger, mostrar logs y health checks.
- Cada integrante deberá poder explicar cualquier parte del código.

Apéndice A: Modelos de Datos (UML)

A continuación se detallan las entidades de cada microservicio con sus campos, tipos y restricciones.

Product

Product
+ Id : Guid // Identificador único
+ Nombre : string // Requerido, máx. 100 caracteres
+ Descripcion : string // Opcional, máx. 500 caracteres
+ Precio : decimal // Requerido, mayor a 0
+ Stock : int // Requerido, mayor o igual a 0
+ Categoria : string // Requerido
+ FechaCreacion : DateTime // Asignado automáticamente al crear

User

User
+ Id : Guid // Identificador único
+ Nombre : string // Requerido
+ Apellido : string // Requerido
+ Email : string // Requerido, único, formato válido
+ PasswordHash : string // Hash de la contraseña (nunca se expone en responses)
+ FechaRegistro : DateTime // Asignado automáticamente al registrar
+ Activo : bool // false cuando el usuario está bloqueado
+ IntentosFallidos : int // Se incrementa en cada login fallido; se resetea al loguearse OK

Order y OrderItem

Order
+ Id : Guid // Identificador único
+ UsuarioId : Guid // Referencia al usuario que realizó la orden
+ Items : OrderItem[] // Lista de productos incluidos en la orden
+ Total : decimal // Calculado automáticamente al crear la orden
+ Estado : string // Pendiente Confirmada Enviada Entregada Cancelada
+ FechaCreacion : DateTime // Asignado automáticamente al crear

OrderItem
+ ProductoId : Guid // Referencia al producto
+ Cantidad : int // Requerido, mayor a 0
+ PrecioUnitario : decimal // Capturado del producto al momento de crear la orden

Cart y CartItem

Cart
+ UsuarioId : Guid // Identificador del usuario dueño del carrito
+ Items : CartItem[] // Lista de productos en el carrito
+ FechaActualizacion : DateTime // Actualizado automáticamente en cada operación

CartItem
+ ProductoId : Guid // Referencia al producto
+ Cantidad : int // Requerido, mayor a 0

Notification

Notification
+ Id : Guid // Identificador único
+ UsuarioId : Guid // Usuario destinatario
+ Mensaje : string // Requerido, máx. 500 caracteres
+ Tipo : string // Email Push SMS
+ Estado : string // Pendiente Enviada Fallida
+ FechaEnvio : DateTime // Asignado automáticamente al registrar

Apéndice B: Ejemplo de `ExceptionHandler` en .NET 8

El siguiente código muestra el patrón recomendado para implementar el manejo de errores en cada microservicio. Está dividido en tres partes: el registro en `Program.cs`, las excepciones de dominio y los handlers.

Registro en `Program.cs`

Aquí se le indica a ASP.NET Core qué handlers usar y en qué orden. El framework los recorre en secuencia y usa el primero que pueda procesar la excepción. Por eso los handlers específicos van antes que el genérico, que actúa como red de seguridad para cualquier error no contemplado.

```
// Program.cs
builder.Services.AddExceptionHandler<NotFoundExceptionHandler>();
builder.Services.AddExceptionHandler<BusinessRuleExceptionHandler>();
builder.Services.AddExceptionHandler<GlobalExceptionHandler>();
builder.Services.AddProblemDetails();

app.UseExceptionHandler();
```

Excepciones de dominio

Son clases simples que extienden `Exception` y agregan el campo `ErrorCode`. Se lanzan desde la capa de servicios cuando ocurre un error de negocio. Cada excepción representa una categoría de error: `NotFoundException` para recursos no encontrados, `BusinessRuleException` para violaciones de reglas de negocio como stock insuficiente o estado de orden inválido.

```
// Excepciones de dominio
public class NotFoundException(string errorCode, string message) : Exception(message)
{
    public string ErrorCode { get; } = errorCode;
}

public class BusinessRuleException(string errorCode, string message) :
    Exception(message)
{
    public string ErrorCode { get; } = errorCode;
}
```

Implementación del handler

Cada `ExceptionHandler` verifica si sabe manejar la excepción recibida. Si no es del tipo esperado devuelve `false` y el framework prueba el siguiente handler. Si la reconoce, construye la respuesta con el formato `Problem Details` incluyendo `errorCode` y `errorMessage` del catálogo.

```
// NotFoundExceptionHandler
public class NotFoundExceptionHandler : IExceptionHandler
{
    public async ValueTask<bool> TryHandleAsync(
        HttpContext context, Exception exception,
```

```

        CancellationToken cancellationToken)
    {
        if (exception is not NotFoundException ex) return false;

        context.Response.StatusCode = 404;
        await context.Response.WriteAsJsonAsync(new
        {
            type    = "https://tools.ietf.org/html/rfc7231#section-6.5.4",
            title   = "Not Found",
            status  = 404,
            detail  = "El recurso solicitado no fue encontrado.",
            instance = context.Request.Path.Value,
            errorCode = ex.ErrorCode,
            errorMessage = ex.Message
        }, cancellationToken: cancellationToken);

        return true;
    }
}

```

Uso desde la capa de servicios

En el Service simplemente se lanza la excepción con el código del catálogo. El handler se encarga del resto; no es necesario ningún try/catch en el controlador.

```

// Uso en la capa de servicio
if (product == null)
    throw new NotFoundException("PRD-001", "Producto no encontrado.");

if (product.Stock < request.Cantidad)
    throw new BusinessException("ORD-005",
        $"Stock insuficiente para '{product.Nombre}'. Disponible: {product.Stock}, solicitado: {request.Cantidad}.");

```